PQC-OnionPKI

A Post-Quantum Public Key Infrastructure Over Tor with Email-Signature Integration

Reproducible Engineering Report and Applied Research Study

Hugo FRANCOIS
Cybersecurity Engineering Student

August 17, 2026

Abstract

The advent of quantum computing threatens widely deployed public-key cryptography such as RSA and ECDSA, impacting the long-term security of authentication and digital signatures. Post-quantum cryptography (PQC) provides new primitives designed to resist quantum adversaries, and is currently being standardized for broad adoption.

However, deploying PQC in real systems is not only a cryptographic problem: it is a systems integration challenge. Trust infrastructures are built around Public Key Infrastructures (PKI), certificate formats, legacy client constraints, and protocol ecosystems that can reject or silently transform non-standard metadata.

This report presents *PQC-OnionPKI*, a post-quantum PKI deployed on a hardened Raspberry Pi and exposed through a Tor onion service. The PKI provides post-quantum signing capabilities and enables a concrete application: signing outgoing emails in a transparent manner, without requiring modifications to mainstream email clients.

Beyond implementation, this work documents the observed structural limits of the current email ecosystem—particularly metadata/header handling—and explains how these constraints shape architectural decisions. The report is designed to be fully reproducible: a reader should be able to rebuild the system from scratch using the provided commands, configurations, and rationale.

Contents

Part I

Introduction and Positioning

Chapter 1

Context, Motivation and Problem Statement

1.1 The Quantum Threat to Contemporary Cryptography

Public-key cryptography underpins modern digital trust infrastructures. Protocols and applications such as TLS, software distribution signatures, and identity systems rely on hardness assumptions (integer factorization, discrete logarithms) that are vulnerable to quantum attacks based on Shor's algorithm.

Although large-scale fault-tolerant quantum computers are not yet available, long-lived authenticity and non-repudiation mechanisms must anticipate future adversaries. In response, post-quantum cryptography (PQC) introduces new primitives intended to remain secure under quantum capabilities, and is being standardized for widespread adoption.

Yet, the availability of PQC algorithms does not imply immediate deployability. Real-world trust systems are built around formats, APIs, and client behaviors that can constrain or delay migration.

1.2 From PQC Algorithms to Post-Quantum PKI

In practice, digital trust is operationalized through a Public Key Infrastructure (PKI). A PKI establishes identities, binds public keys to entities, and defines a chain of trust through certificate authorities (CA), policies, and verification procedures. For end users and applications, PKI is the interface through which cryptographic primitives become usable at scale.

A *post-quantum PKI* raises two classes of challenges:

- **Cryptographic validity:** generating, storing, and using PQC keys and signatures correctly (e.g., ML-DSA / Dilithium-class signatures), including performance and robustness on constrained hardware.
- **Ecosystem compatibility:** integrating PQC identities and proofs into existing software stacks (certificate containers, client libraries, protocols, and user applications) that may not support PQC keys and certificate formats yet.

This project is therefore centered on a post-quantum PKI as the primary artifact. Email signing is treated as an application layer that stresses the PKI under realistic deployment constraints.

1.3 Email Security: A Legacy Ecosystem and a Stress-Test Application

Electronic mail is one of the most widely deployed Internet systems and is deeply rooted in legacy standards prioritizing interoperability and backward compatibility. Modern email security mechanisms include transport protection (TLS), sender authentication (SPF), domain-level signatures (DKIM), and policy enforcement (DMARC).

However, email is also a particularly challenging environment for experimental security extensions:

- messages traverse multiple intermediaries (MTA relays, filtering services),
- metadata can be rewritten, removed, or hidden by providers and clients,
- mainstream clients expose only a subset of headers and may suppress non-standard fields.

For these reasons, email provides a high-value experimental lens: it reveals not only whether PQC signatures can be computed, but whether a PQC-backed proof can be *transported, preserved, and made visible* to end users within today's infrastructure.

1.4 Problem Statement

This work addresses the following applied research question:

> *How far can a post-quantum PKI be integrated into contemporary communication ecosystems, and what structural constraints—formats, intermediaries, and clients—limit the visibility and verifiability of post-quantum proofs in practice?*

A secondary, concrete question is evaluated through the email application:

> *Can outgoing emails be signed with post-quantum signatures in a way that remains compatible with mainstream clients, and what happens to PQC metadata (e.g., custom headers) across real delivery chains?*

1.5 Scope and Experimental Philosophy

PQC-OnionPKI adopts a pragmatic systems approach. The objective is not to redesign standards from scratch, but to maximize what can be achieved *today*, under realistic constraints:

- no modification of mainstream email clients,
- realistic deployment on embedded hardware (Raspberry Pi),
- hardened operational environment and controlled exposure via Tor onion services,
- full reproducibility: commands, configurations, and observed behaviors are documented as first-class results.

Importantly, limitations encountered during integration are treated as results: they inform design decisions and highlight where the ecosystem resists post-quantum extensions. The remainder of this report progressively moves from architecture and hardening to PQC stack deployment, PKI integration, email pipeline engineering, and ecosystem-driven adaptation.

Chapter 2

Contributions and Originality of the Project

This project is positioned at the intersection of post-quantum cryptography, public key infrastructures, and real-world system integration. Its contributions are not limited to the implementation of cryptographic primitives, but rather address the broader question of how post-quantum trust mechanisms interact with existing, large-scale communication ecosystems.

The originality of PQC-OnionPKI lies in its end-to-end experimental scope: from cryptographic foundations to deployment constraints, including system hardening, middleware integration, and client-side behavior observation.

2.1 Primary Contribution: A Practical Post-Quantum PKI

The central contribution of this work is the design and deployment of a functional post-quantum public key infrastructure (PQC-PKI) operating under realistic constraints.

Unlike many theoretical or simulation-based studies, this PKI:

- relies on post-quantum digital signature algorithms selected through the NIST standardization process,
- is deployed on resource-constrained embedded hardware,
- operates as a continuously running trust service rather than an isolated cryptographic experiment,
- exposes real interfaces that can be exercised by external applications.

The PKI is treated as a first-class system component, responsible for key management, signing operations, and trust anchoring. This approach reflects how cryptography is actually consumed in production environments, where primitives are encapsulated within infrastructure services rather than invoked directly by end users.

2.2 Secondary Contribution: Email as a Stress-Test Application

Email signing is deliberately chosen as an application layer not because it is the simplest integration target, but because it is one of the most constrained.

By integrating post-quantum signatures into an SMTP-based delivery pipeline, the project evaluates the behavior of PQC-backed proofs when subjected to:

- legacy protocol semantics,

- multiple intermediary mail transfer agents,
- filtering and policy enforcement mechanisms,
- heterogeneous client implementations with limited metadata exposure.

This application serves as a stress test for the PQC-PKI. It reveals not only whether post-quantum signatures can be computed and attached to messages, but whether such proofs survive transport and remain observable at the user interface level.

2.3 Experimental Observation of Ecosystem Constraints

A key originality of this work lies in its experimental treatment of failure modes. Rather than abstracting away incompatibilities, the project explicitly documents the points at which the current ecosystem resists post-quantum extensions.

These observations include, but are not limited to:

- suppression or invisibility of non-standard SMTP headers in mainstream email clients,
- message reclassification or delivery path alteration triggered by metadata changes,
- divergence between server-side correctness and client-side visibility.

Such behaviors are rarely documented in cryptographic literature, yet they play a decisive role in determining whether a security mechanism is deployable in practice. By treating these constraints as experimental results, the project contributes empirical insight into the gap between cryptographic capability and ecosystem readiness.

2.4 Architectural Adaptation as a Research Result

Another contribution of this work is the explicit linkage between observed constraints and architectural decisions. When initial design assumptions are invalidated by real-world behavior, the system is adapted in a controlled and justified manner.

In particular, the transition from metadata-based proof exposure (SMTP headers) to content-level proof exposure (message body integration) is not presented as a workaround, but as an informed design choice derived from experimental evidence.

This approach aligns with systems research methodologies, where the evolution of an architecture in response to environmental constraints is itself a meaningful outcome.

2.5 Reproducibility and Engineering Transparency

Finally, PQC-OnionPKI contributes a fully reproducible engineering artifact. Every major step of the project is documented with:

- explicit installation procedures,
- concrete configuration files,
- command-level execution traces,
- rationale linking technical choices to encountered constraints.

This level of transparency allows the project to serve both as a research case study and as a practical reference for engineers seeking to explore post-quantum cryptography deployment in constrained or legacy environments.

2.6 Positioning with Respect to Existing Work

While a growing body of literature addresses post-quantum algorithms and their theoretical properties, fewer works examine their integration into operational trust infrastructures. Existing email security mechanisms, such as DKIM or S/MIME, are rarely evaluated under post-quantum assumptions and often assume client-side support that does not yet exist.

By contrast, this project deliberately operates within the boundaries of existing clients and protocols. Its contributions therefore complement algorithmic research by highlighting system-level friction points that must be addressed before post-quantum cryptography can achieve widespread adoption.

2.7 Summary of Contributions

In summary, the contributions of this work can be articulated as follows:

- the design and deployment of a functional post-quantum PKI on realistic hardware,
- the integration of this PKI into a constrained, real-world application (email),
- the experimental identification of ecosystem-level limitations affecting PQC deployment,
- the derivation of architectural adaptations grounded in empirical observation,
- a fully reproducible engineering and research artifact.

These contributions set the stage for the architectural and implementation details presented in the following chapters, beginning with a global view of the system architecture.

Chapter 3

Global System Architecture

This chapter presents a high-level view of the PQC-OnionPKI architecture. The objective is to describe the system components, their interactions, and the trust boundaries that structure the project, without yet entering implementation-specific details. This architectural overview serves as a reference model for the more detailed system, cryptographic, and integration chapters that follow.

3.1 Architectural Overview

PQC-OnionPKI is designed as a service-oriented post-quantum public key infrastructure, deployed on a dedicated embedded platform and consumed transparently by external applications. The system is composed of three main layers:

- a **cryptographic trust layer**, implementing post-quantum key management and signature operations,
- a **system and service layer**, responsible for secure execution, isolation, and exposure of cryptographic services,
- an **application integration layer**, where the PKI is exercised through a real-world use case: email signing.

These layers are intentionally decoupled. This separation allows the PKI to remain application-agnostic, while still being stress-tested by a particularly constrained integration target.

3.2 Core Component: The Post-Quantum PKI

At the center of the architecture lies the post-quantum public key infrastructure itself. Conceptually, the PKI fulfills the same role as a classical PKI: it establishes a root of trust, manages cryptographic keys, and produces verifiable signatures on behalf of identified entities.

In PQC-OnionPKI, the PKI is designed as an active service rather than a static certificate repository. Its responsibilities include:

- generation and storage of post-quantum key material,
- execution of post-quantum signature operations,
- association of signatures with stable identifiers,
- enforcement of operational boundaries between cryptographic material and consuming applications.

Importantly, the PKI is treated as a black-box trust provider from the perspective of applications. Applications do not manipulate post-quantum keys directly; instead, they request signing operations through controlled interfaces.

3.3 System Platform and Execution Environment

The PKI is deployed on a dedicated Raspberry Pi, chosen to reflect a realistic constrained environment rather than a high-performance server. This choice influences architectural decisions in several ways:

- cryptographic operations must remain computationally feasible,
- long-running services must be stable under limited resources,
- security hardening must be compatible with embedded hardware constraints.

The operating system and system configuration are treated as integral parts of the architecture. Hardening measures, service isolation, and access control mechanisms are not considered auxiliary concerns, but essential components of the trust model.

3.4 Exposure Through an Onion Service

To control network exposure while enabling remote access, the PKI services are made available through a Tor onion service. This design choice introduces a clear separation between:

- the internal execution environment of the PKI,
- the external network from which applications interact with it.

The onion service acts as a constrained entry point, reducing the attack surface and decoupling the PKI from direct public IP exposure. From an architectural standpoint, Tor is treated as a transport and isolation mechanism rather than an anonymity feature per se.

3.5 Application Integration Layer

Above the PKI and system layers, the application integration layer represents how post-quantum trust is consumed in practice. In this project, email signing is used as a concrete instantiation of this layer.

The application does not embed post-quantum cryptography internally. Instead, it relies on the PKI to:

- process application-level data,
- produce a cryptographic proof bound to that data,
- return a result that can be attached to the application's output.

This design reflects a realistic deployment model, where applications delegate trust operations to centralized services rather than implementing cryptography locally.

3.6 Trust Boundaries and Threat Model

The architecture defines several explicit trust boundaries:

- between the application and the PKI service,
- between the PKI service and the operating system,
- between the system and the external network.

The PKI platform is assumed to be trusted once properly hardened, while external networks and intermediary services are considered untrusted or partially trusted. Email intermediaries, in particular, are not assumed to preserve or faithfully propagate all metadata.

This threat model directly informs later design decisions, especially regarding how cryptographic proofs are transported and exposed.

3.7 Architectural Rationale

The architecture of PQC-OnionPKI is deliberately conservative. Rather than introducing new protocols or client-side modifications, it seeks to integrate post-quantum trust mechanisms within existing operational boundaries.

This choice reflects the central hypothesis of the project: that the primary obstacle to post-quantum deployment today is not cryptographic feasibility, but ecosystem compatibility. By constraining the architecture to realistic assumptions, the project exposes these obstacles in a measurable and reproducible way.

3.8 Transition to Detailed Design

This chapter has presented the structural blueprint of PQC-OnionPKI. The following chapters progressively refine this blueprint, beginning with the concrete system choices and hardening steps that transform the architecture into a secure, operational platform.

Subsequent chapters will detail how each architectural component is instantiated, configured, and validated, ultimately leading to the integration of post-quantum signatures into a real-world email delivery pipeline.

Chapter 4

Hardware and System Choices

This chapter details the hardware and operating system choices that underpin the PQC-OnionPKI platform. These choices are foundational: they determine the feasibility, security posture, and reproducibility of the post-quantum PKI deployment. Rather than relying on abstract assumptions or simulated environments, the project is grounded in a concrete and fully documented hardware setup.

4.1 Design Objectives and Constraints

The selection of hardware and system components was guided by the following objectives:

- **Realism**: the platform should reflect early real-world deployment conditions rather than idealized laboratory setups;
- **Reproducibility**: all components must be commercially available and easily replaceable;
- **Security**: the system must support meaningful hardening and isolation;
- **Sufficiency**: available resources must sustain post-quantum cryptographic operations.

These objectives intentionally exclude cloud-only or high-performance server infrastructures, which could obscure the practical constraints associated with post-quantum cryptography adoption.

4.2 Embedded Platform Selection

A Raspberry Pi 4 Model B was selected as the execution platform for the PQC-PKI service. This choice is motivated by its widespread availability, mature software ecosystem, and moderate computational constraints.

The platform offers sufficient processing power and memory to execute post-quantum signature algorithms while remaining representative of edge or small-scale trust deployments. Additionally, dedicating a single-purpose device to cryptographic trust operations mirrors established PKI and HSM deployment practices, albeit at a lower assurance level.

4.3 Bill of Materials and Reproducibility

To ensure full reproducibility, all hardware components used in the project are explicitly listed below. The system was assembled exclusively from commercially available components, without modification or proprietary hardware.

The complete hardware setup consists of:

- **Raspberry Pi 4 Model B (4 GB RAM)** — primary execution platform for the PQC-PKI service;
- **Official USB-C Power Supply (15.3 W)** — ensuring stable power delivery under sustained cryptographic load;
- **Active ventilated case with power control** — providing thermal stability and physical power isolation;
- **MicroSD card (32 GB)** — hosting the operating system and application stack;
- **USB 3.0 SD/MicroSD card reader** — used for initial system installation and recovery;
- **Ethernet cable (RJ45, Cat 6)** — wired network connectivity to reduce attack surface and improve reliability;
- **Micro-HDMI to HDMI cable** — local console access during initial setup.

All components were sourced from a single vendor and correspond to a standard Raspberry Pi deployment kit. This configuration avoids reliance on external peripherals beyond what is strictly necessary for system initialization and maintenance. :contentReference[oaicite:0]index=0

4.4 Operating System Selection

The operating system selected for PQC-OnionPKI is a Debian-based Linux distribution specifically tailored for the Raspberry Pi. This choice ensures:

- long-term security updates,
- compatibility with standard Linux hardening mechanisms,
- availability of required toolchains for cryptographic library compilation,
- predictable behavior on embedded hardware.

Using a mainstream distribution reduces integration risk and facilitates independent reproduction of the platform by third parties.

4.5 System Role Separation

The Raspberry Pi is dedicated exclusively to hosting the post-quantum PKI and its supporting services. No unrelated applications or user-facing workloads are deployed on the device.

This strict role separation:

- reduces the attack surface,
- simplifies threat modeling,
- aligns with PKI best practices, where trust anchors are isolated from general-purpose computing tasks.

4.6 Hardware Constraints and Cryptographic Implications

Post-quantum cryptographic primitives are known to impose higher computational and memory demands than classical schemes. Deploying such primitives on embedded hardware introduces measurable constraints, including:

- increased memory usage for key material and signatures,
- longer execution times for signing and verification,
- sensitivity to thermal and power stability.

Rather than treating these factors as limitations to be minimized, the project treats them as experimental variables. Observed performance characteristics are later documented and analyzed as part of the system evaluation.

4.7 Physical Security Assumptions

The platform does not incorporate certified hardware security modules, secure enclaves, or tamper-resistant casing. Physical compromise is therefore considered outside the primary threat model.

This assumption is explicit and intentional. The project does not aim to replicate the guarantees of certified PKI hardware, but to evaluate how far post-quantum trust mechanisms can be deployed using commodity components. The implications of this assumption are discussed later in the context of system limitations and future work.

4.8 Rationale Summary

The hardware and system choices made for PQC-OnionPKI represent a deliberate balance between realism, security, and accessibility. By anchoring the project in a fully documented and reproducible hardware configuration, the resulting observations remain grounded in conditions representative of early post-quantum adoption scenarios.

These choices provide the foundation for the next chapter, which details the initial system installation and configuration steps required to transform this hardware platform into a secure execution environment.

Chapter 5

Initial System Installation and Access Configuration

This chapter documents the initial system installation and access configuration of the PQC-OnionPKI platform. The objective is to establish a precise and reproducible baseline for the operating system, user environment, and access mechanisms, upon which all subsequent hardening and cryptographic integrations are built.

Rather than assuming an abstract system state, this chapter records concrete system properties, observed command outputs, and verified access paths. Where exact historical details cannot be asserted with certainty, this is stated explicitly.

5.1 Operating System Identification

The PQC-OnionPKI platform runs a Debian-based Linux distribution specifically built for Raspberry Pi hardware.

The operating system was identified directly on the running system using standard kernel introspection. The following output was observed:

```
Linux pqc-pi 6.12.47+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64
```

Based on this output, the operating system is formally identified as:

> **Debian GNU/Linux (Trixie-based)**, kernel **6.12.47+rpt-rpi-v8**, architecture **aarch64**, Raspberry Pi build.

This identification is sufficient to characterize the execution environment for all subsequent system and cryptographic operations.

5.2 Operating System Installation Method

The operating system was installed prior to the beginning of this experimental work. The system image corresponds to an official Debian build for Raspberry Pi platforms.

While the exact method used to write the image to the microSD card (e.g., Raspberry Pi Imager, `dd`, or equivalent tooling) cannot be asserted with certainty, this detail does not affect any subsequent configuration or result presented in this report.

For the purposes of reproducibility, the following statement accurately reflects the installation conditions:

> The operating system was installed from an official Debian image for Raspberry Pi. The specific imaging tool used to write the SD card is not critical to the reproducibility of the system configuration and is therefore not detailed further.

5.3 Initial User Account and First Login

The initial user account present on the system is the default Raspberry Pi user:

- **Username**: `pi`

This account is consistently observed across all subsequent interactions, as evidenced by shell prompts of the form:

```
pi@pqc-pi:~ $
```

The first system access was performed locally via a physical console (keyboard and display). This access method served both as the initial login and as a recovery path during later SSH configuration changes.

The initial login sequence involved no specific system commands beyond the standard console authentication process.

5.4 Network Interface and Local Access

The Raspberry Pi is connected to the local network via a wired Ethernet interface (`eth0`). At the time of configuration, the system was assigned the following local IP address:

- **IP address**: `192.168.1.49`

Network interfaces were inspected using the following command:

```
ip a
```

Wired connectivity was intentionally preferred over wireless networking to improve reliability and reduce the exposed attack surface.

5.5 Secure Shell (SSH) Access Configuration

Remote administration of the PQC-OnionPKI platform is performed exclusively via Secure Shell (SSH). SSH access was progressively configured in multiple phases, reflecting increasing security constraints.

5.5.1 SSH Access over the Local Network

During the initial configuration phase, SSH access was enabled over the local network using a non-standard port:

- **SSH port**: 2222

From the client machine (macOS), the following command was used to connect to the system:

```
ssh -p 2222 pi@192.168.1.49
```

On the Raspberry Pi, the SSH daemon was verified to be listening on the expected address and port using:

```
ss -tlnp | grep ssh
```

This command confirmed that the SSH service was bound to the configured port.

5.5.2 SSH Access via Tor Onion Service

To enable secure remote access without direct public exposure, SSH access was later extended through a Tor onion service.

On the client side, Tor availability was verified using:

```
pgrep tor
nc -vz 127.0.0.1 9050
```

The Tor service was managed using the local package manager and service framework. Once Tor connectivity was confirmed, SSH access through the onion service was validated using the following command:

```
ssh -o ProxyCommand="/opt/homebrew/bin/ncat \
--proxy 127.0.0.1:9050 --proxy-type socks5 %h %p" \
pi@<onion_address>
```

For usability and consistency, a client-side shell alias was defined:

```
alias tor-ssh='ssh -o ProxyCommand="/opt/homebrew/bin/ncat \
--proxy 127.0.0.1:9050 --proxy-type socks5 %h %p"'
```

This alias allows SSH connections over Tor to be initiated using:

```
tor-ssh pi@<onion_address>
```

5.6 SSH Key Management

To enforce key-based authentication, multiple SSH key pairs were generated on the client system:

```
ssh-keygen -t ed25519 -C "pqc-onion-client"
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_pqc_backup \
-C "pqc-onion-backup"
```

The public keys were deployed to the Raspberry Pi over the Tor connection using:

```
ssh-copy-id -o ProxyCommand="nc -X 5 -x 127.0.0.1:9050 %h %p" \
pi@<onion_address>
```

Successful key deployment was verified on the Raspberry Pi by inspecting:

```
cat ~/.ssh/authorized_keys
ls -ld ~/.ssh
```

5.7 SSH Server Configuration

The SSH daemon configuration was adjusted to enforce explicit security policies. The configuration file was edited using:

```
sudo nano /etc/ssh/sshd_config
```

The following parameters were set or verified:

- `Port 2222`

- `PermitRootLogin no`
- `PubkeyAuthentication yes`
- `PasswordAuthentication yes`

The SSH service was restarted and verified using:

```
sudo systemctl restart ssh
ss -tlnp | grep ssh
```

5.8 Initial Protective Measures

Basic protective mechanisms were activated early in the setup process to mitigate brute-force and network-based attacks.

Fail2ban was installed and enabled using:

```
sudo apt install fail2ban -y
sudo nano /etc/fail2ban/jail.local
sudo systemctl restart fail2ban
```

The operational status of Fail2ban was verified with:

```
sudo fail2ban-client status
sudo fail2ban-client status sshd
```

Kernel-level hardening parameters were applied using a dedicated sysctl configuration file:

```
sudo nano /etc/sysctl.d/99-pqc-hardening.conf
sudo sysctl --system
```

5.9 Logging and Journal Management

System logging and log rotation capabilities were verified to ensure auditability and long-term stability.

Disk usage of the system journal was inspected using:

```
journalctl --disk-usage
```

Log rotation support was confirmed with:

```
logrotate --version
cat /etc/logrotate.conf
```

5.10 Chapter Summary

At the conclusion of this chapter, the PQC-OnionPKI platform is running a clearly identified and reproducible operating system, with controlled local and remote access paths, enforced SSH security policies, and baseline protective mechanisms in place.

This state forms the trusted foundation upon which deeper system hardening and cryptographic infrastructure deployment are performed in the following chapters.

Chapter 6

System Hardening and Attack Surface Reduction

This chapter details the system hardening measures applied to the PQC-OnionPKI platform. The objective is to reduce the attack surface of the PKI host while preserving operational stability and reproducibility. All configurations described in this chapter are directly derived from the running system and validated through command-level inspection.

Rather than pursuing maximal hardening at the expense of usability or clarity, the approach taken here is conservative, explicit, and auditable. Each measure is motivated by a concrete threat model and verified empirically.

6.1 Firewall Architecture and Network Filtering

Network-level filtering is enforced using `ufw`, which acts as a high-level interface to the underlying `nftables`/`iptables-nft` framework.

The firewall is confirmed to be active:

```
sudo ufw status
```

At the time of inspection, only the following TCP ports are allowed:

- **22/tcp**: legacy SSH (kept temporarily for compatibility),
- **2222/tcp**: primary SSH access port,
- **587/tcp**: SMTP submission (Postfix).

All other incoming traffic is denied by default. IPv6 filtering mirrors the IPv4 policy, ensuring consistent behavior across protocol families.

6.1.1 Low-Level Rule Enforcement

To validate that firewall behavior is effectively enforced at the kernel level, the complete ruleset was inspected using:

```
sudo nft list ruleset
```

The output confirms that:

- the default policy for `INPUT` and `FORWARD` chains is `drop`,
- established and related connections are accepted,

- only explicitly allowed service ports are reachable,
- invalid packets are logged and dropped.

This confirms that `ufw` operates as a policy layer over a correctly enforced packet-filtering backend, rather than as a superficial configuration tool.

6.2 Service Exposure and Runtime Validation

To assess the effective attack surface, all listening sockets were enumerated:

```
sudo ss -tulpen
```

The results show that externally reachable TCP services are strictly limited to:

- `sshd` on port 2222,
- `postfix` on ports 25 and 587.

Other services (e.g., Tor SOCKS proxy on port 9050, CUPS, RPC services) are bound to loopback interfaces or are constrained by firewall rules, preventing external access. This confirms a deliberate separation between internal service dependencies and externally exposed interfaces.

6.3 SSH Hardening and Access Control

Remote administrative access is restricted to SSH, configured with explicit security policies.

The SSH daemon configuration ($/etc/ssh/sshd_config) enforces the following$:

`non-standard listening port (2222),`

`explicit binding to loopback and local LAN addresses,`

`root login disabled (PermitRootLogin no),`

`public-key authentication enabled,`

`keyboard-interactive authentication disabled,`

`PAM enabled for session management.`

The service configuration was validated through both configuration inspection and runtime verification:

```
ss -tlnp | grep ssh
```

This confirms that the SSH daemon is not exposed on wildcard interfaces beyond those explicitly defined.

6.4 Brute-Force Protection with Fail2ban

To mitigate brute-force and credential-stuffing attacks, `Fail2ban` was deployed and configured to protect the SSH service.

The active jail configuration is defined in `/etc/fail2ban/jail.local` and includes:

- monitoring of the SSH service on port 2222,
- a ban time of one hour,

- a detection window of ten minutes,
- a maximum of five failed attempts before banning,
- explicit whitelisting of localhost and the local LAN subnet.

Fail2ban status was verified using:

```
sudo fail2ban-client status
sudo fail2ban-client status sshd
```

At the time of inspection, no active bans were present, indicating both correct operation and absence of attack attempts during the observation period.

6.5 Kernel-Level Hardening via Sysctl

Kernel networking parameters were hardened using a dedicated configuration file:

```
/etc/sysctl.d/99-pqc-hardening.conf
```

The applied measures include:

- reverse-path filtering to mitigate IP spoofing,
- disabling of ICMP redirects,
- disabling of source routing,
- reduction of TCP timestamp exposure to limit fingerprinting.

The configuration was applied and validated using:

```
sudo sysctl --system
```

Observed runtime values confirm that the intended parameters are active. Additional kernel protections inherited from the base distribution (e.g., filesystem hardening, PID limits) complement these custom settings.

6.6 Running Services and Attack Surface Review

To ensure that only required services are active, all running systemd services were enumerated:

```
systemctl list-units --type=service --state=running
```

The result confirms that the system runs a minimal set of services necessary for:

- system operation and device management,
- secure remote access (SSH),
- anonymized transport (Tor),
- email transport and submission (Postfix),
- logging and monitoring.

While some default services (e.g., Avahi, CUPS, Bluetooth) remain enabled, their exposure is constrained by firewall policy and interface binding. Their presence is documented explicitly to preserve transparency and reproducibility.

6.7 Hardening Strategy and Rationale

The hardening strategy adopted in PQC-OnionPKI emphasizes clarity and verifiability over aggressive minimization. Rather than disabling every non-essential service, the system enforces strict network boundaries and access controls, ensuring that dormant services do not translate into externally exploitable vectors.

This approach reflects a realistic operational posture: one where security is enforced through layered controls and continuous validation rather than through assumptions of perfect minimalism.

6.8 Chapter Summary

At the conclusion of this chapter, the PQC-OnionPKI platform operates with:

- a default-deny firewall policy,
- explicitly constrained service exposure,
- hardened SSH access with brute-force protection,
- kernel-level networking hardening,
- auditable and reproducible system state.

This hardened baseline provides the security foundation required for deploying post-quantum cryptographic services and exposing them—selectively and safely—to real-world applications. The following chapters build upon this foundation by introducing cryptographic components and PKI logic into the secured system.

Chapter 7

Secure Remote Access via Tor Onion Services

This chapter describes the integration of Tor onion services into the PQC-OnionPKI platform. The objective is to enable remote administrative access without exposing the system to direct IP-based connectivity, thereby reducing the attack surface while preserving full operational control.

Rather than treating Tor as an auxiliary component, it is positioned here as a core transport layer that shapes both the security model and the architectural constraints of the system.

7.1 Motivation for Onion-Based Access

Exposing cryptographic trust infrastructure directly to the public Internet introduces inherent risks, including network scanning, fingerprinting, and volumetric attacks. These risks are amplified in experimental deployments where post-quantum cryptographic components are evaluated in real-world conditions.

Using a Tor onion service provides:

- implicit network-level anonymity,
- elimination of publicly routable IP exposure,
- resistance to large-scale scanning,
- cryptographic authentication of service endpoints.

This approach aligns with the project's objective of deploying a post-quantum PKI in a constrained yet realistic threat environment.

7.2 Tor Service Configuration Overview

The Tor daemon is configured using the standard system-wide configuration file:

`/etc/tor/torrc`

The platform operates Tor as a client with an embedded onion service, not as a relay or exit node. No relay-specific parameters are enabled.

7.3 Onion Service Definition

The SSH onion service is defined explicitly at the end of the Tor configuration file using the following directives:

```
HiddenServiceDir /var/lib/tor/ssh_hidden_service/
HiddenServicePort 22 127.0.0.1:2222
```

This configuration maps incoming connections on port 22 of the onion service to the local SSH daemon listening on port 2222. The mapping enforces strict separation between external service representation and internal service configuration.

7.4 Hidden Service Material and Permissions

Upon initialization, Tor generates the cryptographic material required for the onion service inside the specified directory. The contents of this directory were inspected using:

```
sudo ls -l /var/lib/tor/ssh_hidden_service/
```

The directory contains:

- the onion service hostname,
- an Ed25519 public key,
- an Ed25519 secret key,
- an authorization directory for client restrictions.

All files are owned by the `debian-tor` user and protected by restrictive permissions. This ensures that onion service credentials are not accessible to non-privileged system users.

7.5 Onion Address Retrieval

The onion service address is generated automatically by Tor and stored locally. It can be retrieved using the following command:

```
sudo cat /var/lib/tor/ssh_hidden_service/hostname
```

For security reasons, the actual onion address is not disclosed in this document and is referenced abstractly as `<onion_address>`.

This command is essential for reproducibility, as it provides the authoritative binding between the Tor configuration and the externally reachable service identifier.

7.6 Tor Service Validation

The operational status of the Tor service was verified using systemd:

```
sudo systemctl status tor@default
```

The service is confirmed to be active and running continuously over multiple days. Runtime logs indicate successful onion service activity, including the reception of rendezvous requests and the establishment of Tor circuits.

These observations confirm that the onion service is not merely configured but actively used and reachable.

7.7 Client-Side Access Model

From the client perspective, SSH access to the platform is performed via the Tor SOCKS proxy. Connections are established without direct knowledge of the server's IP address, relying solely on the onion service identifier.

This access model ensures:

- end-to-end encrypted transport,
- server authentication via onion service keys,
- resistance to network-level interception.

The resulting access path is functionally equivalent to direct SSH while offering substantially improved network-layer security properties.

7.8 Security and Operational Implications

The integration of Tor onion services fundamentally alters the threat model of the PQC-OnionPKI platform. Traditional perimeter defenses based on IP filtering become secondary, while cryptographic identity and service authorization take precedence.

Notably:

- brute-force attacks are mitigated by both Tor circuit behavior and Fail2ban,
- service discovery is restricted to holders of the onion address,
- network-level denial-of-service vectors are significantly constrained.

These properties are particularly relevant when deploying experimental cryptographic services that should remain observable yet controlled.

7.9 Chapter Summary

At the conclusion of this chapter, the PQC-OnionPKI platform exposes its administrative interface exclusively through a Tor onion service, with no dependency on publicly routable IP connectivity.

This design choice provides a robust and reproducible mechanism for secure remote administration and establishes a transport layer consistent with the project's emphasis on cryptographic trust and post-quantum resilience. The following chapters build upon this secure access foundation to introduce the post-quantum cryptographic stack and PKI architecture.

Chapter 8

Cryptographic Environment and Post-Quantum Preparation

This chapter presents the cryptographic environment of the PQC-OnionPKI platform and the preparatory work required to enable post-quantum cryptography within a real-world PKI context. Rather than assuming native post-quantum support, this chapter exposes the structural limitations of contemporary cryptographic toolchains and documents the concrete integration challenges encountered during the project.

This chapter plays a central role in the overall contribution of this work. It highlights the gap between post-quantum cryptographic standardization and its effective deployment in operational infrastructures.

8.1 Baseline Cryptographic Environment

The system relies on OpenSSL as its primary cryptographic framework. At the operating system level, the installed OpenSSL version was identified using:

```
openssl version -a
```

The observed output corresponds to a modern OpenSSL release:

- OpenSSL version: **OpenSSL 3.5.4**
- Platform: **debian-arm64**
- Architecture: **aarch64**

This version represents the default cryptographic stack provided by the operating system and reflects the state of OpenSSL currently deployed on production Linux systems.

However, this baseline environment is not sufficient to support post-quantum cryptographic primitives in a PKI context, as discussed in the following sections.

8.2 Limitations of Native OpenSSL for Post-Quantum Cryptography

Despite its maturity and extensibility, upstream OpenSSL does not natively implement standardized post-quantum cryptographic algorithms such as ML-DSA (Dilithium) or ML-KEM (Kyber).

While OpenSSL 3 introduced a provider-based architecture designed to facilitate extensibility, post-quantum algorithms are not included in the default providers distributed with official OpenSSL releases at the time of this project. As a result:

- post-quantum key generation is unavailable out of the box,
- post-quantum signatures cannot be produced or verified natively,
- X.509 certificates cannot embed post-quantum primitives using standard OpenSSL tooling.

This limitation is not theoretical but structural. OpenSSL prioritizes long-term stability, backward compatibility, and ecosystem-wide interoperability, which constrains the integration of evolving or experimental cryptographic standards.

8.3 liboqs: Post-Quantum Cryptographic Primitives

To address the absence of post-quantum algorithms in OpenSSL, the platform integrates `liboqs`, a cryptographic library developed under the Open Quantum Safe (OQS) initiative.

The presence of `liboqs` on the system was verified using:

```
ls /usr/local/lib | grep oqs
pkg-config --libs liboqs
```

This confirms that `liboqs` is locally installed and available for linking. The library provides implementations of post-quantum algorithms standardized or selected by NIST, including:

- ML-DSA (Dilithium) for digital signatures,
- ML-KEM (Kyber) for key encapsulation,
- SPHINCS+ (analyzed but not retained for the mail use case).

However, `liboqs` operates exclusively at the algorithmic level. It does not provide PKI abstractions, certificate formats, or trust chain management. In particular, it does not support X.509 certificates, certificate authorities, or PKI workflows.

8.4 Role of OpenSSL in the Project

OpenSSL constitutes the foundational cryptographic layer for all standard PKI operations within the project. It provides:

- cryptographic key generation,
- X.509 certificate handling,
- certificate authorities (CA), certificate signing requests (CSR), and signature workflows,
- TLS support and widely interoperable cryptographic formats.

OpenSSL is therefore not merely a cryptographic library, but a complete and mature PKI framework. Without OpenSSL, the construction of an operational, interoperable, and auditable PKI would not be feasible.

8.5 The Structural Gap Between OpenSSL and liboqs

A fundamental mismatch exists between OpenSSL and liboqs:

- OpenSSL provides PKI logic but lacks post-quantum primitives,
- liboqs provides post-quantum primitives but lacks PKI logic.

This structural gap prevents direct deployment of post-quantum cryptography in existing PKI ecosystems. Bridging this gap is mandatory to move from cryptographic research to operational trust infrastructures.

8.6 OpenSSL-OQS as an Integration Layer

To bridge OpenSSL and liboqs, the project relies on OpenSSL-OQS, a fork of OpenSSL extended to support post-quantum algorithms via liboqs.

OpenSSL-OQS was compiled manually, together with liboqs, in order to:

- maintain full control over compilation options,
- guarantee reproducibility of the cryptographic stack,
- expose post-quantum algorithms through the OpenSSL API.

Due to compatibility constraints, OpenSSL-OQS relies on **OpenSSL 1.1.1 (LTS)** rather than OpenSSL 3.x. At the time of the project, OpenSSL versions greater than or equal to 3.x were not yet fully compatible with stable post-quantum provider integration.

This choice is motivated by stability and functional integration, not by obsolescence.

8.7 Hybrid Cryptographic Stack

As a result, the platform operates with a dual cryptographic environment:

- OpenSSL 3.x, provided by the operating system, used for standard system cryptographic needs,
- OpenSSL 1.1.1 + OpenSSL-OQS, used explicitly for post-quantum cryptographic operations.

This hybrid configuration reflects real-world transition strategies currently explored in academic research and pre-industrial deployments. It avoids forcing post-quantum cryptography into incompatible toolchains while preserving interoperability with existing standards.

8.8 Capabilities Enabled by the Integration

Through OpenSSL-OQS, the platform gains the ability to:

- list post-quantum and hybrid algorithms via OpenSSL,
- generate post-quantum key pairs using standard OpenSSL commands,
- perform post-quantum signature and verification operations,
- explore X.509 certificate structures embedding classical, hybrid, or post-quantum keys.

These capabilities were empirically validated through algorithm listings, key generation, signature tests, and performance benchmarks executed on the Raspberry Pi platform.

8.9 Rationale for the Chosen Approach

The choice to integrate OpenSSL and liboqs through OpenSSL-OQS follows a pragmatic and forward-looking rationale:

- compatibility with existing PKI ecosystems,
- reuse of standardized formats such as X.509 and CSR,
- support for hybrid cryptographic models during transition phases,
- alignment with NIST post-quantum standardization efforts and emerging regulatory frameworks.

This approach reflects the realistic transition path currently adopted in post-quantum research and early deployment efforts, rather than an isolated experimental setup.

8.10 Chapter Summary

At the conclusion of this chapter, the PQC-OnionPKI platform operates with:

- a modern system-provided OpenSSL installation,
- a dedicated post-quantum cryptographic stack based on OpenSSL 1.1.1 and OpenSSL-OQS,
- a clear separation between cryptographic primitives and PKI logic,
- a hybrid model enabling progressive post-quantum integration.

These elements define the cryptographic boundary conditions of the project. The following chapter builds upon this foundation to introduce the architecture of the post-quantum public key infrastructure deployed on the platform.

Chapter 9

Post-Quantum Public Key Infrastructure Architecture

This chapter presents the public key infrastructure (PKI) architecture designed and implemented within the OnionPQC-PKI project. The objective of this chapter is not to describe low-level implementation details, but to formalize the architectural choices that enable post-quantum cryptography to be integrated into a real-world messaging environment.

A central contribution of this work is the explicit distinction between two PKI models: a fully post-quantum infrastructure implemented for experimental and research purposes, and a hybrid infrastructure currently deployed to ensure operational compatibility with existing systems. This distinction is fundamental and is therefore introduced explicitly at the architectural level.

9.1 PKI Design Objectives

The PKI architecture of OnionPQC-PKI is guided by the following objectives:

- enable post-quantum cryptographic security for message authenticity,
- preserve compatibility with existing operating systems, mail clients, and protocols,
- remain minimal, auditable, and reproducible,
- support a progressive transition from classical to post-quantum cryptography.

Rather than attempting to replace the existing PKI ecosystem abruptly, the project explores realistic transition paths aligned with current research and pre-industrial practices.

9.2 Overall PKI Structure

The PKI is structured as a simple hierarchical infrastructure:

Root CA → Intermediate CA (Mail) → Application Certificates

This hierarchy mirrors conventional PKI designs while remaining sufficiently minimal to support experimentation, reproducibility, and security analysis. The intermediate CA is dedicated to mail-related trust domains, allowing separation of concerns between global trust anchors and application-specific usage.

9.3 Two PKI Models: Experimental vs Operational

To address both research goals and deployment constraints, two distinct PKI models were implemented within the project.

9.3.1 Full Post-Quantum PKI (Experimental)

A first version of the infrastructure was implemented as a fully post-quantum PKI. In this configuration:

- the Root CA uses post-quantum signature algorithms (ML-DSA / Dilithium),
- the intermediate Mail CA also relies exclusively on post-quantum cryptography,
- end-entity certificates are post-quantum native,
- the entire chain is generated using OpenSSL-OQS linked with liboqs.

This infrastructure was successfully generated and validated from a cryptographic perspective. Key generation, certificate issuance, and signature verification were tested and confirmed functional.

However, this full post-quantum PKI is not used in the operational mail pipeline.

Deployment Limitation Current operating systems and mail ecosystems do not support post-quantum X.509 certificates. In particular:

- macOS keychains do not accept post-quantum certificates,
- SMTP/TLS authentication mechanisms expect classical algorithms,
- mainstream mail clients do not recognize post-quantum trust chains.

As a result, a fully post-quantum PKI cannot be integrated transparently into the mail stack. This limitation is structural and ecosystem-wide, not implementation-specific.

Role in the Project The full post-quantum PKI is therefore retained as:

- a proof of cryptographic feasibility,
- a research artefact demonstrating end-to-end PQC trust chains,
- a reference point for future migration once ecosystem support matures.

9.3.2 Hybrid PKI (Operational)

The PKI model currently deployed in OnionPQC-PKI is a hybrid architecture combining classical compatibility and post-quantum security.

In this configuration:

- the Root CA uses classical cryptography (RSA or ECDSA),
- the intermediate Mail CA is also classical,
- device authentication (SMTP/TLS) relies on classical certificates fully compatible with macOS and standard mail clients,
- message authenticity is ensured through an independent post-quantum signature mechanism.

The post-quantum signature does not rely on X.509 client certificates. Instead, it is applied directly to the mail content and injected as an application-level cryptographic proof.

Key Property This design ensures that:

- the mail pipeline remains fully functional on existing infrastructure,
- post-quantum security is applied where it is most critical: message integrity and authenticity,
- the absence of PQC support in clients does not invalidate the cryptographic protection.

9.4 Effective PKI State and Trust Separation

The effective PKI deployed on the platform follows a strict separation of trust domains and responsibilities.

9.4.1 PKI Directory Structure

The implemented PKI follows the structure below:

```
~/OnionPQC-PKI/pki/
 root/
    certs/
       root.crt
       root.srl
    private/
        root.key

 mail/
    certs/
       mail_ca.crt
       mail_ca.srl
    newcerts/
    private/
       mail_ca.key
    mail_ca.csr
    mail_ca_ext.cnf

 mail-signer/
     certs/
     private/
     mail_sign.csr
     mail_sign_ext.cnf
```

9.4.2 Security Properties

The PKI enforces the following security constraints:

- private keys are stored outside application directories,
- strict file permissions are applied (mode 600),
- clear separation exists between:
  - Root CA,
  - Mail CA,

– mail signing entity.

This separation prepares the infrastructure for future execution by dedicated services, such as SMTP filters or isolated signing processes.

9.5 Role of the OnionPQC-PKI Server

The OnionPQC-PKI server acts as the trust enforcement point of the architecture. It hosts:

- the PKI hierarchy,
- the post-quantum signing keys,
- the logic required to generate cryptographic proofs for outgoing messages.

Clients do not perform post-quantum cryptographic operations themselves. Instead, the server ensures that each outgoing message receives a post-quantum signature before being relayed.

This server-centric design avoids modifications to client software while preserving cryptographic guarantees.

9.6 Architectural Positioning and Transition Strategy

The architecture deliberately decouples:

- classical PKI for transport-layer compatibility,
- post-quantum cryptography for message authenticity.

This separation enables a progressive migration path. When operating systems and mail ecosystems eventually support post-quantum X.509 certificates, the hybrid PKI can evolve toward a fully post-quantum infrastructure without redesigning the mail pipeline.

9.7 Chapter Summary

This chapter formalized the PKI architecture underlying the OnionPQC-PKI project. Two PKI models were clearly distinguished: a fully post-quantum infrastructure implemented for research purposes, and a hybrid infrastructure currently deployed for operational compatibility.

By separating classical trust anchors from post-quantum message authentication, the project demonstrates a realistic and deployable approach to post-quantum transition in existing communication systems.

The following chapter builds upon this architectural foundation to describe how post-quantum cryptographic proofs are applied to electronic mail messages.

Chapter 10

Cryptographic Validation

This chapter presents the cryptographic validation phase of the OnionPQC-PKI project. It details the generation of post-quantum keys, the signing and verification processes, and the performance benchmarks conducted on the Raspberry Pi platform. The objective is to demonstrate both the functional correctness and the practical feasibility of deploying post-quantum cryptography in a constrained, real-world environment.

10.1 Key Generation

The cryptographic foundation of the system relies on the ML-DSA-65 signature scheme, corresponding to Dilithium3 as standardized by NIST. Key generation is performed directly on the Raspberry Pi, using the OpenSSL-OQS integration to ensure compatibility with standardized cryptographic interfaces.

The following command was used to generate the signing key pair:

```
openssl genpkey -algorithm ML-DSA-65 \
  -out mail_sign.key
```

The public key was then extracted as follows:

```
openssl pkey -in mail_sign.key -pubout -out mail_sign.pub
```

The private key is stored securely on the server and is never exposed to client devices. It is used exclusively by the OnionPQC-PKI signing service and is not associated with any X.509 identity or TLS authentication mechanism.

Key generation was observed to complete successfully without errors, confirming the correct installation and operation of the OpenSSL-OQS provider on the Raspberry Pi.

10.2 Message Signing

Message signing is performed server-side during mail processing. The signed object is not the raw email content, but a canonicalized representation composed of selected headers and the normalized body.

Specifically, the signed data includes:

- All standard email headers except custom `X-PQC-*` headers
- The normalized textual body of the email
- A deterministic concatenation format to ensure reproducibility

The canonicalized message is hashed using SHA-256 prior to signing. The resulting hash is then signed using the ML-DSA-65 private key.

The signing operation can be reproduced manually using the following commands:

```
openssl dgst -sha256 -binary canonical_message.txt > msg.hash

openssl pkeyutl -sign \
  -inkey mail_sign.key \
  -in msg.hash \
  -out signature.bin
```

The generated signature is then encoded and embedded into the outgoing email, either as structured headers or as a cryptographic footer, depending on the delivery constraints imposed by downstream mail servers.

10.3 Signature Verification

Verification follows the inverse process of signing and can be performed independently by any recipient possessing the public key.

The verification steps are as follows:

1. Reconstruct the canonical message using the same normalization rules
2. Compute the SHA-256 hash
3. Verify the signature using the public key

The verification command is:

```
openssl dgst -sha256 -binary canonical_message.txt > msg.hash

openssl pkeyutl -verify \
  -pubin \
  -inkey mail_sign.pub \
  -sigfile signature.bin \
  -in msg.hash
```

A successful verification produces the following output:

```
Signature Verified Successfully
```

Any modification to the message body or headers results in verification failure, confirming the integrity guarantees provided by the scheme.

10.4 Performance Benchmarks on Raspberry Pi

To evaluate the practical viability of post-quantum cryptography on constrained hardware, benchmark tests were conducted directly on a Raspberry Pi 4.

The following operations were measured:

- Key generation
- Signature creation
- Signature verification

Each operation was executed multiple times, and average execution times were recorded. The observed benchmark results are summarized below:

Operation	Average Time (seconds)
Key Generation	∼0.019
Signature	∼0.020
Verification	∼0.019

These results demonstrate that ML-DSA-65 operations can be executed in approximately 20 milliseconds on commodity ARM hardware.

10.5 Interpretation of Results

The benchmark results confirm several important points.

First, the performance overhead of post-quantum signatures is compatible with real-time email processing. Even under high mail throughput, the measured latency remains negligible compared to network and SMTP processing delays.

Second, the symmetry between signing and verification times indicates a balanced computational cost, which is advantageous for scalable verification by recipients.

Finally, the successful deployment and execution of ML-DSA-65 on a Raspberry Pi validate the feasibility of post-quantum cryptographic infrastructures on low-cost, low-power devices.

These results support the architectural choice of centralizing post-quantum signing operations on a dedicated server, while keeping client-side requirements minimal. They also confirm that post-quantum cryptography can be integrated into existing communication protocols without prohibitive performance penalties.

This validation phase provides a solid experimental foundation for the subsequent deployment and operational phases of the OnionPQC-PKI system.

Chapter 11

Selected SMTP Architecture

This chapter describes the SMTP architecture retained for the OnionPQC-PKI project. It presents a concise reminder of SMTP operation, explains the role of Postfix within the system, details the possible integration points for cryptographic processing, and justifies the precise injection point selected for post-quantum signature integration.

The architectural choices presented in this chapter are driven by real-world constraints observed during experimentation with modern mail transfer agents and client software.

11.1 SMTP Operation Overview

The Simple Mail Transfer Protocol (SMTP) is a store-and-forward protocol designed for reliable email delivery across heterogeneous networks. In a standard deployment, an email is transmitted from a Mail User Agent (MUA) to a Mail Transfer Agent (MTA), which is responsible for relaying the message toward the destination domain.

SMTP processing typically involves the following stages:

- Message submission by a client
- Reception and queuing by an MTA
- Optional filtering and content processing
- Outbound relay toward the next-hop MTA

Modern SMTP infrastructures often involve multiple MTAs, each potentially modifying or filtering messages according to policy, security requirements, or compatibility constraints.

11.2 Role of Postfix in OnionPQC-PKI

In the OnionPQC-PKI architecture, the Raspberry Pi does not replace existing mail providers such as Gmail or Outlook. Instead, it operates as an intermediate outbound SMTP relay.

The functional role of the Raspberry Pi is strictly limited to:

- Intercepting outgoing emails
- Applying post-quantum cryptographic processing
- Relaying the processed message to the Internet

The resulting mail flow is as follows:

```
Client (Mac / iPhone / Mail app)
        ↓
SMTP Provider (Gmail / Outlook)
        ↓
OnionPQC-PKI (Postfix on Raspberry Pi)
        ↓
Destination SMTP / Recipient MTA
```

In this design, client devices never connect directly to the Raspberry Pi. The Pi is positioned as a trusted outbound relay, ensuring that cryptographic processing remains centralized and independent from client-side capabilities.

This architecture was validated using the following Postfix inspection commands:

```
postconf | grep relayhost
postconf | grep myhostname
postconf | grep mydestination
```

The output confirms that the Raspberry Pi is configured as a relay host and does not act as a final mail destination.

11.3 Postfix Processing Pipeline

Postfix processes messages through a modular pipeline composed of distinct stages. After message reception, content may be passed through optional filters before final delivery or relay.

Several potential injection points exist within the Postfix pipeline:

- Header and body checks
- Cleanup service hooks
- Milter interfaces
- Content filters (pipe-based)

Each of these mechanisms offers different trade-offs in terms of control, stability, and compatibility.

11.4 Selected Injection Point for PQC Signing

The OnionPQC-PKI project integrates post-quantum signature generation using a Postfix `content_filter`. This filter is applied after SMTP message reception and before outbound relay.

The signing step is implemented as a pipe-based filter executed during Postfix message processing. The filter receives the message via standard input and outputs the processed message via standard output.

The precise integration statement can be summarized as follows:

> *The PQC signing step is injected during the Postfix content filtering stage, after message reception and before outbound SMTP relay, ensuring that the signed message corresponds exactly to the server-emitted content.*

The filter is implemented by the script `pqc_sign_mail.py`, executed under a dedicated unprivileged user.

Script name	`pqc_sign_mail.py`
Path	`/usr/local/bin/pqc_sign_mail.py`
Execution user	`pqc`
Mode	SMTP content filter (STDIN → STDOUT)

The content filter configuration was verified using the following commands:

```
sudo postconf | grep content_filter
sudo cat /etc/postfix/master.cf
ps aux | grep pqc_sign_mail
```

The relevant `master.cf` excerpt is shown below:

```
smtp      inet  n  -  y  -  -  smtpd
  -o content_filter=pqcfilter

pqcfilter unix  -  n  n  -  10 pipe
  flags=Rq user=pqc argv=/usr/local/bin/pqc_sign_mail.py
```

This configuration ensures that all outgoing messages processed by Postfix are passed through the PQC signing filter.

11.5 Signed and Unsigned Message Components

The message subject to cryptographic protection is a canonicalized representation composed of:

- All standard email headers, excluding custom `X-PQC-*` headers
- The normalized textual body of the message

Multipart messages are not currently supported and are explicitly excluded from the signing process.

The following elements are not included in the cryptographic signature:

- `X-PQC-*` headers
- The post-signature cryptographic footer

The footer is intentionally injected after the signature operation. It serves as a visible proof for end users and does not participate in the cryptographic integrity guarantee. This separation is a deliberate architectural choice.

11.6 Justification of the Selected Design

The content filter stage was selected as the unique injection point satisfying all project constraints.

The primary reasons are:

- Maximum compatibility with Gmail, Outlook, and mainstream MTAs
- Avoidance of header suppression or rewriting
- Stability of the signed message content
- Full server-side control of the cryptographic process

- Simplified debugging and traceability
- Strict adherence to the SMTP processing model

No alternative injection point provides equivalent guarantees in terms of message stability and reproducibility.

11.7 Operational Constraints and Observations

During experimentation, several structural limitations of the email ecosystem were observed:

- Custom `X-*` headers are frequently hidden or removed by MTAs and mail clients
- Some MTAs may modify message bodies during relay
- SPF, DKIM, and DMARC policies impose strict constraints on message handling
- Mainstream mail clients do not provide native support for post-quantum cryptography

These constraints directly motivated the adoption of a hybrid architecture:

- Classical cryptography for transport-level security (TLS)
- Post-quantum cryptography for content-level authenticity
- A visible footer to convey cryptographic assurance to users

This design reflects a pragmatic integration of post-quantum security within the realities of existing email infrastructures.

11.8 Concrete Verification Example

To illustrate the operational behavior of the selected SMTP architecture, this section presents a concrete verification example performed directly on the OnionPQC-PKI server.

After message processing by Postfix and injection of the post-quantum signature, the resulting email is stored in RFC 5322 format and can be verified offline using the dedicated verification script.

11.8.1 Offline Signature Inspection

The following command was executed on the Raspberry Pi to inspect a signed email message:

```
sudo python3 pqc_verify_mail.py signed.eml
```

The script extracts and displays the post-quantum signature metadata embedded in the message headers. A truncated excerpt of the output is shown below:

```
X-PQC-Alg: ML-DSA-65
X-PQC-Hash: sha256
X-PQC-Signature: =?utf-8?q?G6nsOfLjQ2MjIsm5yZTYLusDOoF+CzsGcPeBaANw+k1bpwHqH?=
 =?utf-8?q?4bu03T+KYWg6AiC/i2dwA+dFn67lwGS84yBz2NgOTFNuIxN1hdigemlX+b42xka9+?=
 =?utf-8?q?+g9AhJFdoGpb2oT1KYmGlYZHpURgnk7e1/4uZ/LOKvkI/H0bYJQbCPcLUthwj0HeO?=
 =?utf-8?q?5jrchroX79oDqE54cfZ56Ny1MQQR3oTrFJJAeT8XfybXfc94Gx12i37hjs6Hm0GHF?=
 =?utf-8?q?2HBBjB9UJy66Iyv4ctIAB83FF8abPFWSrQywDOWhpLhhJruikuySEgtTVC3vSxX67?=
 =?utf-8?q?sW0Pr7LzKUYjEFEobNJtkQPMY2zUMOdAwOWz2sYaCtqe+bGCv/P8Qjlcobsgxu MOM?=
 =?utf-8?q?WD9IdtTLf+jXTjHClWXzD7qDkxAvX1lxFyGHKMNDlQdN+YXSN6kylrtil8GtczDDf?=
```

=?utf-8?q?VAqmqYYzudFeuxSc66cH929M8aFS45OSKe1xFylP1FznHKDafE+I2EUeLQ8RlwdZz?=
=?utf-8?q?hPTOCX/dOaFB+CH+A0A1nVAH+k3jlyV9PYdMG/irhhZKfXbSGqwuGigLkCvamT6vG?=
=?utf-8?q?5+oHlTBcA2IXYKcLRfevQp07k2MKU/BHyPJO0to81MjziQ+wqQZ6Ufr2eKnBrfoSl?=
=?utf-8?q?/AkfZTRetTtEwIvetRLj8DelUm+sox7JDx/g/aCmzlzSowRgXvgXe22eoaB5B5MoA?=
=?utf-8?q?fLUHl3vamCmnA2KlisvMxvW4rwUTN0I0I/VewowlzaH31CW7227x8+2+k7M6y6y2j?=
=?utf-8?q?T8i+pjbpmt1n6RgDJQ2fDlNGgoZ8oGVbvXMvlg4k/bbJPI1LoJAzIZNpBZue4DkO9?=
=?utf-8?q?Q+xrniKjKfdcQ+lYgqIjOkI4w9GGXm3k5KyoedFAaMt79T4MlHhu2NZqLeKoM0W5r?=
=?utf-8?q?hGLCBzSVva+se06epvsxER2ooynCpk8KDZmxlO9M45f1HKqSRoltHBnuBWMGvKIIx?=
=?utf-8?q?IK9fFz3VHFXXESFbZa1K6PqdxFEzjVuqrheo4IRQWAgnHM+5aWlYYqD8ZoW8YhFH0?=
=?utf-8?q?SMoPsQ3TPhN/FxjK1rUMycE9HizR4Y0xKe7lG1v7GWLqlzGfesPvUqhMdHlWojftz?=
=?utf-8?q?OEdUEQ6n/5ce7EdmZBLmCErLvQEpTfgcTaWj48btl8HaZiQexshpCIoJvfxScNkGg?=
=?utf-8?q?Q4txdgkP/u/VNUzrB1PZWNbS0oCI9IHSL5nx3qqXhwSfsAbtVbx4J43WtDn2kGlgz?=
=?utf-8?q?KF0lIGLgNRjlXZu00kWWZjlHvK8bhrr4dtYWcrSAbaYQ/yMuJMWaZ8wARC9BzBQXn?=
=?utf-8?q?f9ERkFpF+XYLDmH7dMh681FA3Kw+HIuYyKbRzzjbMytz6fIvBLt3LlcSCWsEgac+V?=
=?utf-8?q?/qdRgt80Bj712rzBxnPERZ+i4vq34Cvs+o+Bjy2yBlXjvXqjhJuWZLmiOhqWDReLe?=
=?utf-8?q?i98sBAeJ+AkpSV6EdTzwlw1XmoXUxM7gJf5HU2SVcaMI8SEz60/+5oSP6uHRMx/Bs?=
=?utf-8?q?ZV8AI14ovaFzoXANiv09n4jWeBsi6MtL3QPVHf1epkHyrZWzACPdppN3oJ+Y1EqNw?=
=?utf-8?q?4hgTfalNuwQOwmD2rxDHzeDG4vPY8CzYq+BXmh8bj6ilpvEGN+vJmpt2TTfRCHh0X?=
=?utf-8?q?HqSF5rJQBOZRmC4koOB7V9E+U5IrUYgp3Bce3dVIiWZmXD4dXs0ahcjmXHLIy7x14?=
=?utf-8?q?J7RC6OZn/cQQU0IxBE+gZEsJOHO6krNev578eVMO38SGpSrjKtAb7NR/UI8uIKFQ9?=
=?utf-8?q?14n6sCPT5DseTjiA9QwltRTYRHjYfLASY//mWKJccY5pe71Cxgkb5kYeu3u0fukot?=
=?utf-8?q?GpYqA76NdfcwA24+r4YhXSTMnJzY22HaqYJoRDycf8rO6OBGt7wTza+YPfdEtoZgh?=
=?utf-8?q?6KPfTKElcz0Jd74X7BgG8pfPyBu5MdU//Z7W6i/Rzulncj387otZpdlumB1h80+Xh?=
=?utf-8?q?1td6V+JPoAXU7nRzeenycXuDxVmx9bQ5P7vNweRclHR5D6H4b7cN9VhUGfj6C4wNx?=
=?utf-8?q?JudmOoFptCeLzNPOGWT3zVF7VvRwGiPibqhQSA9rWAJOzcDSOAZKIV1WqsRngW2+X?=
=?utf-8?q?AzIPVF391tvoe8VV2tdd2lZKXiGEkqLciDIo86JUbYDRsUae5yCXJIG0pPgA7PbfH?=
=?utf-8?q?AUHoSmIvk9RGIOOy+0nq5hVz0t3RGvIRosD9N1pKSv5QsV/VMgAM6YCSbmr7dCvbs?=
=?utf-8?q?JCx2UFYNe1KaM39rrZoau3+bdWcMS+wly/gMwjef/Ld4drDPLRSX+LUZcU8BseYhk?=
=?utf-8?q?onsYuIUvvVSQapz4zOtSc3QiHy+9qF5gj5qRS8w0v0IaU9JAHwCQTskDcb1jZWJaB?=
=?utf-8?q?HGh507kbQFAlZHKaUbIJXro7i/ZmNDVpbemJQahvck4e5u4jNWnemfDNyHqn9tAQP?=
=?utf-8?q?lRrkk5Kre7WUCzPBXhCS/LHP2a8435VDLn6BHZqSDZnzKVgD2SqAiWftq3E8mM+Iy?=
=?utf-8?q?E2AZQiv0bRqEE7hcdc88Y8+bu/qgry+rFMtMesAhh0PxK+FyAz5yErRmnfitIK487?=
=?utf-8?q?2s5otivBci/Cw49G6nVVGX8QJ8kXISz5Gi1o1beH+AW8Dw2GFPyIF4Ge+ngtQbpcf?=
=?utf-8?q?OzCtxnDMVDSrEQuX+BciEhaGFEVtGFnHSCMS2bBM/xImx15dVZBls1XNVAIXXZHdf?=
=?utf-8?q?583PKx/jq5RsXiva4BIxuWeVZID3/pAqtKrsswa/0U4eymYHwKJ7pJL2FF2EY/08y?=
=?utf-8?q?P6/aMwOEZzeFvkdPYAUEb4z1ciTSJ6yzeLVZoJGKeqWOv4H13PysyJ65gi4bRHfMn?=
=?utf-8?q?jzYcmHt8Va7vGsgo/z2bFAfb2aHaSH/wONcNUI9cxZGdCGDjLkfrTNKz3VyefVENi?=
=?utf-8?q?BcXRJBYjZYuV6rJd/W566fMEFwmvV2JubTpwFl+MLlj0xCNXPwwGlH/9AxqyqOGdR?=
=?utf-8?q?GAJpByTibOIvXsmaMbvpD+2obZJ/lz/6FO7ieP5Gc9F9CV5X1iv2r6Y339xNtzEG3?=
=?utf-8?q?8DT5yPAFXuR+W3yRJv5NoW7qweB1MpbBg90OUr51UfhDZ0k5eYh+eAIykDoEN3vl5?=
=?utf-8?q?mSJUamqDkaTF/akDOnabamtZCgBVnXGSwvWxyxpZZvHKSsjC7+05Et93CCLkkR64R?=
=?utf-8?q?mtkSZ4A1JeHtVeAIkxtZy3rnaZ4EUI8Psi02WMeAG6epOksaEcDCh8gfpRqSOX2fQ?=
=?utf-8?q?lKt/1SGeQWAoq7RVYArGLC5aKBYw8qGpASFu7GsGOpWdMwADQwQqZC8fO/kiO8R3P?=
=?utf-8?q?52iBCjp8Royf6012dsYi8IvYiebuNZjTweF0Kwtw9nDJft14MHCApEcHGQzTuJBGk?=
=?utf-8?q?Y/abhtAw9pIqNJyHjTzyC8C8aIK4yjxVP1QE9fJR5LEI98UfRzS2oDnl7m0nmrnQF?=
=?utf-8?q?VdUtoKAKTIPkVsaQwHJYWH9ut7OL168do+H7X7VVEpkfl/jle8xcl/18soiflW6I/?=
=?utf-8?q?E3pCZGn3DbBZGIcXm3nO6PPQ9OqxitYmV8QdL++4Q5kvZA3qKIFOPQhS/JE4CPk5p?=
=?utf-8?q?Cw/IHF0BtWOQ+v0UPS6ZffdLnKK+sqixezM8MJ+e+SAsbDFg/wcnvecmxbIBLEAKT?=
=?utf-8?q?89oa3JbdseBHnJJ3+RkW9fw8xObDSzkm7hMNrWZuuEDVagRLI9ElJiAbFVnDrEAYn?=
=?utf-8?q?Ups39Z1xYe8yKIR07LIQY7Sdu9inLBUXJUh9NAN/zeQSta2wD17mGjLN10vPMGBo2?=
=?utf-8?q?7LyfgtkTYjPZd2r+wkKdq9y+agE3cx7KCqhqODZ/NnbXTqXZDjPeeHr8BUjpHyQJi?=

```
 =?utf-8?q?wLLTn6gcoy/v7ZJS3jF9CniHrEyc3aML9WRs9Rp9pLZ+raeAkvDU1dD2DYg08yz9r?=
 =?utf-8?q?7PU4lcNCqnKvthWnuoDXuSMwKUbdJaPtmULvaz6ohwXw+Ashuhtt+CQzMz7zbn52a?=
 =?utf-8?q?CTiWY4gQ3ALoj+Fx8IfBpcNx2XOS5RxLPJF2GRK10Sy2ItcWciZZf+hh9L6sy1Su+?=
 =?utf-8?q?HQwF/C3QylTMwtojklYefnMcO+j2fUF6MHJrvlhzIaoDDkYPx7ggR0eVO6kzCdCJc?=
 =?utf-8?q?YMRHgzWUPn99YF2VZVx9kEgeYfIzThmAOl2XHXJhIMCJiBSHENT6HG7kpx1TJRrRP?=
 =?utf-8?q?tekkOTaDJBboglXrRYxFFW0AAnTp7TjO93nJm9jDFCTYWs7g4MiSR1A/YdUkwxw2l?=
 =?utf-8?q?VM6vdzP2I+hmopCfe9yHITh3BCKdx+FO4GtYF2Ax77geWyI2O+xFEXaIH/sp+uEQJ?=
 =?utf-8?q?lIoOoyVPViQuTkKb5lPcIaETNdVCAtqO2XVVD9ykY3FSYHivm36NmLemRyMDJqKIq?=
 =?utf-8?q?2Qgbht7H//WOwAMrLCuWHg4zfeGutjBt2wDny7KNBzFYbhYGb3NsH+oPsmKWmn/FF?=
 =?utf-8?q?gb3IURIwOFiAnLUfEOPcEGjM+S5+iqNvdDF9jmcHO6vn+AAAAAAAAAAAAAAAAAAAA?=
 =?utf-8?q?BQkMFR8o?=
X-PQC-KeyId: =?utf-8?b?QTU6QjE6QTQ6MDY6NOU6MjM6NkY6QTU6MkU6MUY6NjI6NkU6NkM6?=
X-PQC-Processed: yes
```

The presence of these headers confirms that the message was processed by the OnionPQC-PKI content filter and that post-quantum signing was successfully applied.

11.8.2 Verification Scope

The verification script performs the following operations:

- Extraction of `X-PQC-*` metadata
- Reconstruction of the canonicalized message
- SHA-256 hashing of the canonical representation
- Verification of the ML-DSA-65 signature using the public key

The body of the message remains readable and unmodified from the user perspective:

```
Hello PQC Onion PKI
```

This confirms that cryptographic processing does not interfere with message usability.

11.8.3 Selective Header Inspection

To explicitly demonstrate that the PQC-related headers are present at the server level, the following filtered command was used:

```
sudo python3 pqc_verify_mail.py signed.eml | grep X-PQC
```

Result:

```
X-PQC-Alg: ML-DSA-65
X-PQC-Hash: sha256
X-PQC-Signature: =?utf-8?q?G6nsOfLjQ2MjIsm5yZTYLusDOoF+CzsGcPeBaANw+k1bpwHqH?=
X-PQC-KeyId: =?utf-8?b?QTU6QjE6QTQ6MDY6NOU6MjM6NkY6QTU6MkU6MUY6NjI6NkU6NkM6?=
X-PQC-Processed: yes
```

This demonstrates that the cryptographic proof is reliably embedded at the SMTP layer, even if such headers are not displayed by mainstream email clients.

11.8.4 Architectural Implications

This example highlights a fundamental property of the selected architecture: the cryptographic validity of a message can be verified independently of client-side behavior.

Even when `X-PQC-*` headers are hidden or stripped by downstream MTAs or MUAs, the server-side verification remains possible, reproducible, and deterministic.

This observation directly justifies:

- the choice of Postfix content filtering as the injection point,
- the separation between cryptographic proof and user-visible footer,
- the hybrid design combining classical transport security with post-quantum content authentication.

The example confirms that OnionPQC-PKI provides a verifiable cryptographic trust layer at the SMTP level, independent of the constraints imposed by the current email ecosystem.

11.9 Conclusion

The selected SMTP architecture enables reliable, reproducible post-quantum signature integration without requiring modifications to client devices or external mail providers. By leveraging Postfix content filtering, OnionPQC-PKI achieves a clean separation between transport, cryptographic processing, and user-visible proof, while remaining compatible with the current email ecosystem.

This architecture establishes a robust foundation for the operational deployment of post-quantum cryptographic services in real-world communication systems.

Chapter 12

Postfix Configuration and Traceability

This chapter documents the Postfix configuration used in the OnionPQC-PKI project. It focuses on configuration traceability, the precise parameters modified, the role of each directive in the SMTP processing pipeline, and the functional validation of the setup.

The objective is to provide a reproducible and auditable description of the mail transfer configuration supporting post-quantum signature integration.

12.1 Modified Postfix Configuration Files

The Postfix installation is based on the default Debian configuration. Only two configuration files were manually modified to integrate the PQC signing mechanism.

File	Status	Role
`/etc/postfix/main.cf`	Modified	PQC filter integration, SMTP routing
`/etc/postfix/master.cf`	Modified	Declaration of the `pqcfilter` service

No additional Postfix mechanisms were used. In particular, the following features were explicitly not deployed:

- `milter`
- `cleanup` hooks
- `transport_maps`
- `sender_dependent_relayhost_maps`
- TLS policy maps

File modification metadata was verified using standard filesystem inspection commands:

```
sudo ls -l /etc/postfix/
sudo stat /etc/postfix/main.cf
sudo stat /etc/postfix/master.cf
```

The timestamps confirm manual modifications aligned with the PQC integration phase.

12.2 Postfix Parameters Modified in `main.cf`

Only parameters with a functional or security impact were modified. All non-default parameters were extracted using the following command:

```
sudo postconf -n
```

The most critical directive for PQC integration is:

```
content_filter = pqcfilter
```

This directive instructs Postfix to pass outgoing messages through a dedicated content filter before relay.

Additional relevant parameters observed in the configuration include:

- `inet_interfaces = all`
  Controls network exposure of the SMTP service.
- `mynetworks = 127.0.0.0/8 192.168.1.0/24`
  Restricts relay permissions to trusted local networks.
- `relayhost =`
  Indicates that the system performs direct outbound relay.
- `smtpd_tls_security_level = encrypt`
  Enforces TLS for incoming SMTP connections.

These directives collectively define the Raspberry Pi as a controlled outbound SMTP relay with enforced transport security and minimal exposure.

12.3 Custom Service Definition in `master.cf`

Post-quantum signing is implemented through a custom Postfix service declared in `master.cf`.

The following service was added manually:

```
pqcfilter unix  -  n  n  -  10  pipe
  flags=Rq user=pqc argv=/usr/local/bin/pqc_sign_mail.py
```

The characteristics of this service are summarized below:

Service type	`unix`
Execution mode	`pipe`
Execution user	`pqc`
Input	SMTP message (STDIN)
Output	Modified message (STDOUT)
Role	Post-quantum message signing

This service is invoked automatically for outgoing messages by the `content_filter` directive.

No other standard Postfix services (such as `submission` or `smtpd`) were modified beyond the filter attachment.

12.4 External Scripts Used by Postfix

Two scripts are involved in the PQC mail processing workflow.

Script	Location	Role	User
`pqc_sign_mail.py`	`/var/lib/pqc/pqc-mail/`	PQC signing	`pqc`
`pqc_verify_mail.py`	Project workspace	Offline verification	`pi`

Only `pqc_sign_mail.py` is executed by Postfix. The verification script is used exclusively for testing and validation outside the MTA.

Script presence and permissions were verified using:

```
sudo ls -l /var/lib/pqc/pqc-mail/
ps aux | grep pqc_sign_mail
```

Strict file permissions ensure that private key material remains accessible only to the dedicated `pqc` user.

12.5 Functional Validation and Tests

The correct operation of the Postfix configuration was validated through multiple tests.

12.5.1 Mail Processing Test

A test email was sent through the SMTP pipeline. Postfix logs confirm that the message was passed to the PQC filter and signed successfully:

```
pqc-mail: mail signed successfully
postfix/pipe: relay=pqcfilter, status=sent
```

This confirms correct invocation of the content filter.

12.5.2 Log Inspection

Operational logs were inspected using:

```
sudo journalctl -u postfix -n 50
```

The logs show:

- successful execution of the PQC signing script,
- message removal from the queue after processing,
- no delivery failures related to the filter.

Warnings related to overridden TLS parameters were observed but do not affect the functional correctness of the PQC integration.

12.5.3 Port and Service Verification

Active Postfix services and listening ports were verified using:

```
ss -tulpen | grep postfix
```

The system correctly listens on SMTP ports (25 and 587), consistent with its role as a relay and submission endpoint.

12.6 Design Constraints and Configuration Choices

Several deliberate design decisions guided the Postfix configuration:

- `content_filter` was selected for stability and auditability.
- `milter` was avoided due to increased complexity and dependency overhead.
- Cleanup hooks were not used to preserve message determinism.
- Default Debian Postfix behavior was preserved whenever possible.

These choices reflect a balance between security, simplicity, and reproducibility.

12.7 Conclusion

The Postfix configuration deployed in OnionPQC-PKI provides a minimal, auditable, and effective integration point for post-quantum cryptographic processing. By limiting modifications to essential configuration files and leveraging standard Postfix mechanisms, the system achieves reliable message signing while maintaining compatibility with the existing email ecosystem.

This configuration forms the operational backbone of the PQC-enabled SMTP pipeline.

Chapter 13

Post-Quantum Signature Pipeline

This chapter describes the exact post-quantum signature pipeline implemented in the OnionPQC-PKI project. It constitutes the cryptographic core of the system and strictly reflects the behavior of the production SMTP content filter.

The scope of this chapter is intentionally limited to:

- the Python signing script,
- message parsing,
- canonicalisation,
- hashing,
- post-quantum signature generation,
- injection of cryptographic metadata into the message.

No consideration related to client-side rendering, user experience, SMTP ecosystem, or future evolution is addressed here.

13.1 Post-Quantum Signing Pipeline Overview

The post-quantum signing process is implemented as a synchronous SMTP content filter. The pipeline is composed of the following sequential processing blocks.

Postfix
Outgoing RFC 5322 message

Input Channel
STDIN (pipe)

`pqc_sign_mail.py`
Postfix content_filter (executed as user `pqc`)

Step 1 — Message Parsing

- Full RFC 5322 message
- Headers and body
- UTF-8 decoding
- CRLF normalized to LF

Step 2 — Canonicalisation

- All headers except `X-PQC-*`
- Original header order preserved
- `text/plain` body only
- Separator: `\n\n`
- Final newline enforced

Step 3 — Hashing (OpenSSL internal)

- Algorithm: SHA-256
- Input: raw canonical message
- No explicit Python hash computation

Step 4 — Post-Quantum Signature

- Algorithm: ML-DSA-65 (Dilithium3)
- Provider: OpenSSL-OQS
- Private key: `mail_sign.key`
- Output: binary signature

Step 5 — Encoding

- Binary signature encoded in Base64

Step 6 — Metadata Injection

- `X-PQC-Alg`
- `X-PQC-Hash`
- `X-PQC-Signature`
- `X-PQC-KeyId`
- `X-PQC-Processed`
- RFC 2047 encoding and folding

Output Channel
STDOUT (pipe)

This diagram represents the complete and authoritative processing flow executed for each outgoing message.

13.2 Signing Script Integration

13.2.1 Script Characteristics

The entire pipeline is implemented in a single Python script:

Script name	`pqc_sign_mail.py`
Location	`/var/lib/pqc/pqc-mail/`
Execution user	`pqc`
Invocation mode	STDIN → STDOUT
Integration point	Postfix `content_filter`
Role	Canonicalisation, signing, injection

The script is monolithic by design. No external helper scripts or cryptographic wrappers are involved in the signing path, which simplifies auditing and ensures a reduced attack surface.

13.3 Message Parsing

The script receives a complete RFC 5322 message via standard input. Parsing is performed using the Python `email` module with the default policy:

```
msg = message_from_file(sys.stdin, policy=default)
```

This parsing step ensures:

- full header and body extraction,
- UTF-8 decoding,
- normalization of CRLF line endings into LF.

The resulting message object serves as the sole input for canonicalisation.

13.4 Deterministic Canonicalisation

Canonicalisation defines the unique message representation covered by the cryptographic signature. The process is fully deterministic.

13.4.1 Canonicalisation Function

```
def canonicalize(msg):
    headers = []
    for k, v in msg.items():
        if not k.startswith("X-PQC-"):
            headers.append(f"{k}: {v.strip()}")

    body = msg.get_body(preferencelist=("plain",))
    body_text = body.get_content().replace("\r\n", "\n").rstrip()

    return "\n".join(headers) + "\n\n" + body_text + "\n"
```

13.4.2 Header Canonicalisation Rules

- All headers are included except those prefixed with `X-PQC-*`.
- Header order is preserved exactly.
- Header field names and casing are preserved.
- Header values are stripped of surrounding whitespace.
- Header unfolding is handled implicitly by the email parser.
- No sorting or reordering is applied.

13.4.3 Body Canonicalisation Rules

- Only `text/plain` bodies are processed.
- Multipart messages are not supported.
- CRLF sequences are converted to LF.
- Trailing whitespace is removed.
- UTF-8 encoding is enforced.
- A final newline is appended explicitly.

13.4.4 Canonical Message Structure

```
Header1: value
Header2: value
...
HeaderN: value

Body
```

This structure uniquely defines the signed message for a given input.

13.5 Hashing Strategy

No explicit hash computation is performed in Python. Hashing is delegated to OpenSSL as part of the signature operation.

```
HASH = "sha256"
```

The canonical message is passed directly to OpenSSL, which applies SHA-256 internally before signature generation. This ensures consistency with OpenSSL-OQS behavior and avoids redundant hashing logic.

13.6 Post-Quantum Signature Generation

The signature algorithm used is ML-DSA-65 (Dilithium3), provided by the OpenSSL-OQS provider.

The exact command executed is:

```
openssl dgst \
  -provider oqs \
  -provider default \
  -sign /var/lib/pqc/pqc-mail/pki/mail-signer/private/mail_sign.key
```

This operation:

- computes SHA-256 internally,
- generates a post-quantum signature,
- outputs a raw binary signature.

The resulting signature is Base64-encoded prior to injection:

```
sig_b64 = base64.b64encode(signature).decode()
```

13.7 Injection of Cryptographic Metadata

After successful signature generation, cryptographic metadata is injected directly into the message as RFC 5322 headers.

The following headers are added:

- `X-PQC-Alg: ML-DSA-65`
- `X-PQC-Hash: sha256`
- `X-PQC-Signature: <Base64-encoded signature>`
- `X-PQC-KeyId: <SHA-256 certificate fingerprint>`
- `X-PQC-Processed: yes`

These headers:

- are injected only after successful signature generation,
- are part of the final RFC 5322 message,
- constitute the exclusive cryptographic proof produced by the pipeline.

Header encoding and line folding comply with RFC 2047 and are handled automatically by the email library.

13.8 Idempotence and Error Handling

The pipeline is designed to be idempotent and fail-open.

If the `X-PQC-Processed` header is already present, the message is relayed unchanged and no cryptographic operation is performed.

In case of error:

- the message is always relayed,
- no message is blocked,
- an error indicator may be injected.

Possible error indicators include:

```
Authentication-Results-PQC: pqc=temperror (sign-timeout)
Authentication-Results-PQC: pqc=temperror (sign-error)
```

Operational events are logged via syslog under the `pqc-mail` identifier.

13.9 Conclusion

This chapter documented the complete and exact post-quantum signature pipeline implemented in OnionPQC-PKI. The pipeline combines deterministic canonicalisation, standardized hashing, and post-quantum cryptography within a minimal and reproducible SMTP integration.

All cryptographic guarantees provided by the system are materialized exclusively through the injected `X-PQC-*` headers, which fully represent the signed state of the message.

Chapter 14

Injection des headers PQC

This chapter addresses a major technical issue encountered during the deployment of the OnionPQC-PKI project: the injection, structure, and verification of post-quantum cryptographic metadata through custom SMTP headers.

Unlike classical email security mechanisms such as DKIM or S/MIME, the proposed design relies exclusively on RFC-compliant custom headers to transport the post-quantum signature and its associated verification metadata. The objective of this chapter is to document precisely the headers used in production, their exact format, and the server-side verification process.

No consideration related to client-side rendering or user interface behavior is addressed in this chapter.

14.1 Headers Defined in Production

The post-quantum signature pipeline injects a fixed and strictly controlled set of custom headers. No additional headers are generated dynamically, and no standard authentication headers are modified.

Table 14.1 lists the exact headers used in production, including their activation conditions.

Header	Presence	Condition
X-PQC-Alg	Always	Successful signature
X-PQC-Hash	Always	Successful signature
X-PQC-Signature	Always	Successful signature
X-PQC-KeyId	Always	Successful signature
X-PQC-Processed	Always	Success or error
Authentication-Results-PQC	Conditional	Error only

Table 14.1: Post-quantum headers injected in production

No other PQC-related headers are generated by the system.

14.2 Header Format and Encoding

Each header follows a strictly defined format aligned with RFC 5322 and RFC 2047 requirements. The encoding strategy is determined by the nature and length of the underlying data.

14.2.1 Algorithm and Hash Identification

`X-PQC-Alg` identifies the post-quantum signature algorithm used. Its value is a fixed ASCII string:

```
X-PQC-Alg: ML-DSA-65
```

`X-PQC-Hash` specifies the cryptographic hash function implicitly applied by OpenSSL during the signing operation:

```
X-PQC-Hash: sha256
```

Both headers are ASCII-encoded and never folded.

14.2.2 Cryptographic Signature

`X-PQC-Signature` carries the post-quantum signature generated by the ML-DSA-65 algorithm. The signature is produced in binary form, then Base64 encoded prior to injection.

Due to its length, this header is automatically encoded and folded according to RFC 2047:

```
X-PQC-Signature: =?utf-8?q?Dblykz4dlSwScVkEcgh4B++a4OUzRUUUJPMAiMNw2yUpxN...?=
```

The folding process is handled transparently by the Python email library.

14.2.3 Key Identifier

`X-PQC-KeyId` uniquely identifies the signing key through the SHA-256 fingerprint of the X.509 certificate associated with the mail signer.

The value is injected using RFC 2047 encoding and may be folded automatically:

```
X-PQC-KeyId: =?utf-8?b?QTU6QjE6QTQ6MDY6N0U6MjM6NkY6QTU6MkU6MUY6NjI6NkU6NkM6...?=
```

14.2.4 Processing Marker

`X-PQC-Processed` is a logical marker used to ensure idempotence within the SMTP pipeline:

```
X-PQC-Processed: yes
```

This header prevents recursive processing and guarantees that each message is handled exactly once.

14.2.5 Authentication Results

`Authentication-Results-PQC` is injected only in error scenarios. It encodes the verification outcome using a normative textual format:

```
Authentication-Results-PQC: pqc=pass (ml-dsa-65)
```

Possible values include:

- `pqc=pass (ml-dsa-65)`
- `pqc=fail (bad-signature)`
- `pqc=fail (invalid-chain)`
- `pqc=temperror (sign-error)`
- `pqc=temperror (sign-timeout)`

14.3 Real Message Example

Listing 14.3 shows an excerpt of a real message as produced by the production pipeline after successful signing.

```
From: alice@example.com
To: bob@example.com
Subject: Test PQC Mail
Date: Sat, 20 Dec 2025 18:00:00 +0000
X-PQC-Alg: ML-DSA-65
X-PQC-Hash: sha256
X-PQC-Signature: =?utf-8?q?Dblykz4dlSwScVkEcgh4B++a4OUzRUUUJPMAiMNw2yUpxN...?=
X-PQC-KeyId: =?utf-8?b?QTU6QjE6QTQ6MDY6NOU6MjM6NkY6QTU6MkU6MUY6NjI6NkU6NkM6...?=
X-PQC-Processed: yes

Hello its Hugo,
This mail is signed with PQC.
```

14.4 Server-Side Verification

Verification is performed using the `pqc_verify_mail.py` script. The script reconstructs the canonical representation of the message, decodes the signature, and verifies it cryptographically.

The verification process includes:

- Presence of `X-PQC-Signature`
- Valid Base64 decoding
- Deterministic canonicalisation
- Post-quantum signature verification (ML-DSA-65)
- Full PKI chain validation (Root CA → Mail CA → Mail Signer)

The script can be invoked either on a file or via standard input:

```
./pqc_verify_mail.py signed.eml
cat signed.eml | ./pqc_verify_mail.py
```

14.5 Injection Position and Guarantees

Header injection is performed strictly after cryptographic processing and before the message is returned to Postfix via standard output.

This ordering guarantees that:

- The signed message representation is immutable
- Cryptographic metadata does not affect the signed content
- Verification remains stable and reproducible
- No recursive modification occurs

The post-quantum signature is therefore injected as a set of RFC-compliant custom headers after cryptographic processing and before reinjection into the SMTP pipeline, ensuring a stable and verifiable message representation independent of transport-layer mechanisms.

Chapter 15

Observed Behavior of Email Clients

This chapter presents the experimental observations related to the behavior of mainstream email clients when receiving messages signed by the OnionPQC-PKI pipeline. The objective is strictly empirical: to document what is effectively visible to end users once a post-quantum signed message reaches common mail clients.

The cryptographic validity of the signature and its server-side verification have been established in previous chapters. This chapter focuses exclusively on client-side reception and header visibility.

15.1 Test Environment

All experiments were conducted using real email accounts and real message deliveries. Two distinct recipient domains were involved:

- A personal Gmail account (`@gmail.com`)
- An institutional account (`@student-cs.fr`)

No Google Workspace, Microsoft 365, or iCloud Mail enterprise environments were used. The use of two independent domains excludes any intra-domain filtering artifact.

15.2 Email Clients Tested

The following clients were effectively tested and observed:

- Gmail Web (desktop browser)
- Gmail Mobile (iOS and Android)
- Apple Mail on macOS
- Outlook Web
- Outlook Desktop

Apple Mail was identified through message headers as:

```
Mime-Version: 1.0 (Mac OS X Mail 16.0 (3864.200.81.1.6))
```

All observations are supported by server logs, raw `.eml` files, and client-side captures.

15.3 Experimental Methodology

Messages were sent exclusively via SMTP using the configured Postfix relay and the PQC content filter. No webmail interface was used for message submission.

The methodology was identical for all tests:

- Single message source
- Plain text content (`text/plain`)
- No attachments
- Mandatory passage through Postfix and the PQC signing pipeline

Server-side confirmation of successful signing was systematically observed:

```
pqc-mail: mail signed successfully
```

15.4 Observed Behavior: Gmail Web

In Gmail Web (desktop), both the standard message view and the *Show Original* interface were examined.

The following headers were present:

- From
- To
- Subject
- Date
- Message-ID
- Content-Type
- Content-Transfer-Encoding
- Mime-Version
- X-Universally-Unique-Identifier

The following headers were entirely absent:

- X-PQC-Alg
- X-PQC-Hash
- X-PQC-Signature
- X-PQC-KeyId
- X-PQC-Processed

No warning, error message, or security indicator was displayed. The absence persisted even in the raw message source.

15.5 Observed Behavior: Apple Mail on macOS

Apple Mail exhibited the same behavior. Standard headers were visible, including `Message-ID` and `X-Universally-Unique-Identifier`, but no PQC headers were accessible.

Neither the graphical interface nor the raw header inspection revealed any `X-PQC-*` fields.

15.6 Web Versus Desktop Comparison

No observable difference was detected between web-based and desktop-based clients. Table 15.1 summarizes the findings.

Client	PQC Headers Visible	Raw Source Access
Gmail Web	No	No
Apple Mail macOS	No	No
Outlook Web	No	No
Outlook Desktop	No	No

Table 15.1: Visibility of PQC headers across clients

15.7 Inter-Domain Observations

Messages sent from Gmail to the external `student-cs.fr` domain exhibited the same behavior:

- Successful delivery
- Confirmed PQC signature on the server
- Complete absence of PQC headers on the client

This demonstrates that the phenomenon is independent of the recipient domain.

15.8 Impact of SPF, DKIM, and DMARC

Classical authentication headers related to SPF, DKIM, and DMARC were consistently present and correctly populated.

No correlation was observed between the outcome of these mechanisms and the presence or absence of PQC headers. The disappearance of `X-PQC-*` headers cannot be explained by standard email authentication policies.

15.9 Evidence and Reproducibility

All observations are supported by:

- Raw signed `.eml` files on the server
- Postfix and application logs
- Client-side captures of header views
- Direct inspection using:

```
cat tests/signed.eml
grep X-PQC tests/signed.eml
journalctl | grep pqc-mail
```

The behavior was reproducible across multiple tests and clients.

15.10 Experimental Conclusion

The post-quantum signature is correctly generated, injected, and verifiable on the server side. However, the corresponding custom headers do not reach the user interface of major email clients, including advanced header inspection views.

This behavior is reproducible, client-independent, domain-independent, and does not trigger any client-side warning or error.

Chapter 16

Structural Analysis of the Header Visibility Problem

This chapter provides a structural and protocol-level analysis of the systematic invisibility of the `X-PQC-*` headers on the client side, despite their correct injection, transport, and cryptographic validity on the server side.

Rather than identifying a local misconfiguration or implementation flaw, this analysis aims to explain why the observed behavior is inherent to the current email ecosystem and therefore unavoidable under present conditions.

16.1 Empirical Restatement of the Observed Phenomenon

The previous chapters established the following facts in a reproducible and experimentally validated manner:

- the `X-PQC-*` headers are correctly injected by the SMTP content filter,
- they are present in the final RFC 5322 message on the server side,
- they are verifiable using an independent verification script,
- they are systematically absent from all tested mail clients,
- no error or warning is reported to the recipient,
- the behavior is independent of:
  - the mail client,
  - the access mode (web or desktop),
  - the destination domain.

These observations rule out application-level bugs, misconfiguration of Postfix, or incorrect header injection. The issue must therefore be analyzed at a structural level.

16.2 Role of SPF, DKIM, and DMARC

SPF, DKIM, and DMARC are often suspected when message modifications are observed. A rigorous analysis shows that none of these mechanisms explains the disappearance of the PQC headers.

16.2.1 SPF

SPF exclusively validates the sending IP address against the domain's policy. It does not inspect, alter, or filter message headers or body content. Consequently, SPF cannot be responsible for the removal or masking of `X-PQC-*` headers.

16.2.2 DKIM

DKIM signs a predefined subset of headers explicitly selected at signing time. Custom headers with the `X-*` prefix are not included by default and are not required for DKIM validation.

As a result, the removal of `X-PQC-*` headers does not invalidate DKIM signatures and is fully compliant with the DKIM model.

16.2.3 DMARC

DMARC enforces alignment policies between SPF and DKIM and defines delivery actions (none, quarantine, reject). It does not mandate header preservation nor affect client-side header visibility.

The absence of any DMARC-related warning confirms that the observed behavior is considered acceptable within the standard email authentication framework.

16.3 Modern MTA Policies and Message Normalization

Large-scale mail providers operate multi-stage Mail Transfer Agent (MTA) pipelines that include normalization, security filtering, and presentation-layer processing.

Within this architecture, non-standard headers are commonly treated as optional metadata. Their retention is not guaranteed, especially when they are not required for message rendering or security enforcement.

The systematic disappearance of `X-PQC-*` headers is therefore consistent with current MTA design practices.

16.4 Filtering of Non-Standard Headers

The `X-` prefix was historically intended for experimental extensions. However, modern mail infrastructures impose no obligation to preserve or expose such headers.

In practice:

- no transport-level guarantee exists for `X-*` headers,
- no standard mandates their visibility to end users,
- MTAs are free to remove, internalize, or suppress them.

The experimental results indicate that suppression is the default behavior across all tested clients.

16.5 Transport vs Presentation Separation

A fundamental aspect of modern email systems is the strict separation between message transport and message presentation.

Mail clients no longer display a faithful representation of the RFC 5322 message. Instead, they expose a curated view controlled by the provider, even when advanced options such as "view original message" are used.

As a consequence, the absence of PQC headers from the user interface does not imply their absence from the transport pipeline.

16.6 Why the Behavior Is Inevitable Today

The inevitability of the observed behavior is explained by four structural factors:

1. **Lack of Standardization**: no RFC currently defines post-quantum email signatures or standardized PQC headers.
2. **Centralized MTA Control**: large providers actively normalize and sanitize messages before client delivery.
3. **Operational Security Priorities**: minimizing hidden metadata and non-essential channels reduces attack surface.
4. **Decoupling of Cryptography and UX**: cryptographic validity does not imply user-visible indicators.

Under these conditions, any solution relying exclusively on custom headers is structurally non-observable at the client level.

16.7 Scientific Implications

This result does not indicate a failure of the post-quantum signature pipeline. Instead, it reveals intrinsic limitations of the current email architecture.

The experiments demonstrate that:

- post-quantum cryptography can be integrated at the SMTP level,
- cryptographic proofs can be generated and verified correctly,
- the existing email model is not designed to expose non-standard cryptographic evidence to users.

This positions the project within a research and systems-analysis perspective rather than a purely engineering one.

16.8 Conclusion

The disappearance of `X-PQC-*` headers on the client side is neither a bug nor a misconfiguration.

It is the direct consequence of:

- historical SMTP design choices,
- the absence of PQC-related email standards,
- modern MTA security policies,
- and the separation between transport and presentation layers.

This chapter establishes a rigorous explanatory framework showing that a cryptographically valid post-quantum signature may be fully functional while remaining invisible to end users in today's email ecosystem.